

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЁТ

По лабораторной работе №3

По курсу «Разработка кроссплатформенных приложений»
на тему «Обработка исключительных ситуаций»

Выполнили
студенты группы 23ВВИ2:

Родионов А. А.

Пырков Д. А.

Приняли:
Юрова О.В.
Карамышева Н.С.

Пенза 2026

Цель работы: изучить механизм обработки исключительных ситуаций.

Лабораторное задание: модифицировать приложение из предыдущей лабораторной работы, реализовав проверку вводимых данных с использованием механизма исключений. Необходимо создать свой класс, унаследованный от класса `Exception`, и генерировать исключение, если возникает попытка создать экземпляр класса `RecIntegral` со значениями, не являющимися числами в диапазоне от 0,000001 до 1000000. В качестве обработки исключения необходимо выводить диалог, содержащий предупреждение о некорректности введенных данных.

Ход работы:

1. В среде NetBeans был создан класс `InvalidDataException` являющийся наследником стандартного класса `Exception`. Данный класс предназначен для сигнализации о попытке использования числовых значений, выходящих за пределы допустимого диапазона от 0,000001 до 1 000 000.

Класс `InvalidDataException` содержит конструктор, принимающий имя параметра и его значение, на основании которых формируется информативное сообщение об ошибке, передаваемое в конструктор родительского класса `Exception` через вызов `super(...)`. Это сообщение впоследствии доступно через унаследованный метод `getMessage()` и используется при отображении диалогового окна пользователю.

Помимо конструктора, класс содержит статический метод `validate`, который инкапсулирует логику проверки. Метод принимает строковое наименование проверяемого параметра и его числовое значение типа `double`. Если значение является нечисловым (NaN), бесконечным (Infinite), меньше нижней границы диапазона (0,000001) или превышает верхнюю границу (1 000 000), метод создаёт и выбрасывает экземпляр `InvalidDataException` с соответствующим описанием. В противном случае метод завершается без побочных эффектов, что означает успешное прохождение проверки.

Конструктор класса `RecIntegral` объявлен с ключевым словом `throws InvalidDataException`, что обязывает вызывающий код либо обрабатывать данное исключение, либо передавать его далее по стеку вызовов. В теле конструктора последовательно вызывается метод `InvalidDataException.validate` для каждого из трёх параметров: верхнего предела, нижнего предела и шага интегрирования. Если хотя бы одно значение не проходит проверку, создание объекта прерывается выбросом исключения, и ссылка на некорректный экземпляр никогда не будет возвращена вызывающему коду.

Аналогичная проверка реализована в каждом из сеттеров класса — `setUpperLimit`, `setLowerLimit` и `setStep`. Вызов `InvalidDataException.validate` осуществляется до присвоения нового значения полю объекта. Таким образом, если передаваемое значение находится вне допустимого диапазона, поле сохраняет своё предыдущее корректное значение, а вызывающий код получает исключение для обработки.

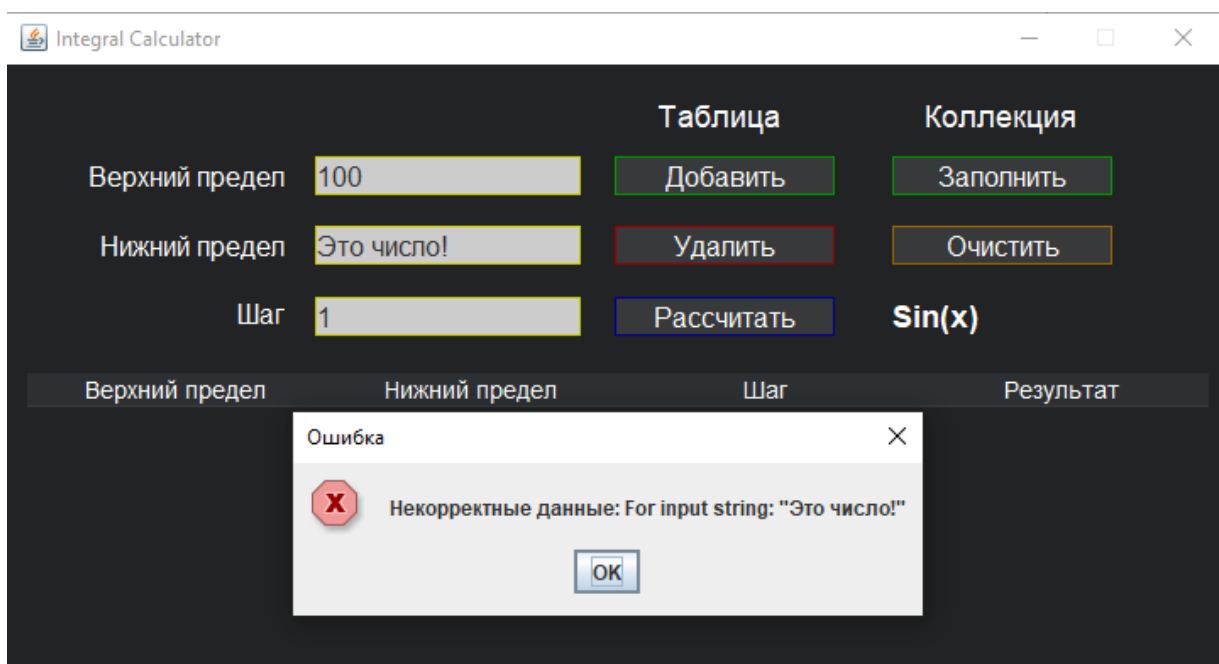
В методе `ButtonAddActionPerformed`, отвечающем за добавление новой записи в таблицу, вызов конструктора `RecIntegral` помещён внутрь блока `try`. Блок `catch (InvalidDataException e)` перехватывает исключение в случае, если пользователь ввёл значение вне допустимого диапазона. В качестве реакции на исключение вызывается метод `JOptionPane.showMessageDialog`, отображающий модальное диалоговое окно с типом `WARNING_MESSAGE`. Текст предупреждения извлекается из объекта исключения посредством метода `getMessage()`, что позволяет пользователю видеть, какой именно параметр содержит некорректное значение и каков допустимый диапазон. Запись в таблицу и в список данных при этом не добавляется, поскольку выполнение кода после конструктора прерывается переходом в блок `catch`. Блок `catch (NumberFormatException e)` сохранён для перехвата ситуаций, когда введённая строка не может быть преобразована в число вовсе.

В слушателе `TableModelListener`, установленном на модель таблицы в конструкторе формы, вызовы сеттеров класса `RecIntegral` также обёрнуты в блок `try`. При перехвате `InvalidDataException` отображается аналогичное

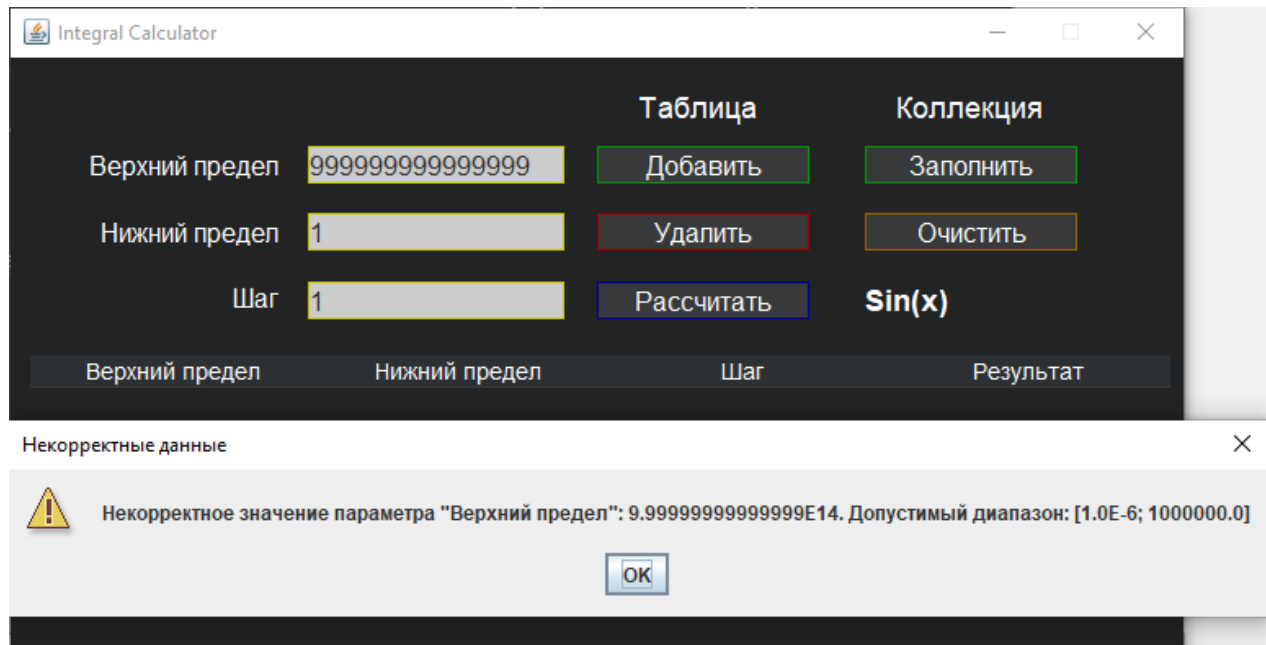
диалоговое окно с предупреждением, после чего в ячейку таблицы возвращается предыдущее корректное значение соответствующего поля объекта `RecIntegral`. Это достигается повторным вызовом `model.setValueAt` с результатом соответствующего геттера. Благодаря тому, что сеттер не изменяет поле объекта при невалидном значении, геттер гарантированно возвращает последнее корректное значение, обеспечивая согласованность данных между моделью таблицы и внутренним списком объектов.

При вводе некорректного значения последовательность событий выглядит следующим образом. Пользователь вводит значение и инициирует действие. Вызывается конструктор или сеттер класса `RecIntegral`, который делегирует проверку статическому методу `InvalidDataException.validate`. Метод `validate` обнаруживает нарушение допустимого диапазона и выполняет оператор `throw`, создавая экземпляр `InvalidDataException`. Исключение поднимается по стеку вызовов из метода `validate` через конструктор или сеттер `RecIntegral` в метод-обработчик событий класса `mainForm`, где перехватывается соответствующим блоком `catch`. В блоке `catch` извлекается сообщение исключения и отображается в модальном диалоговом окне, информирующем пользователя о необходимости ввести значение в допустимом диапазоне.

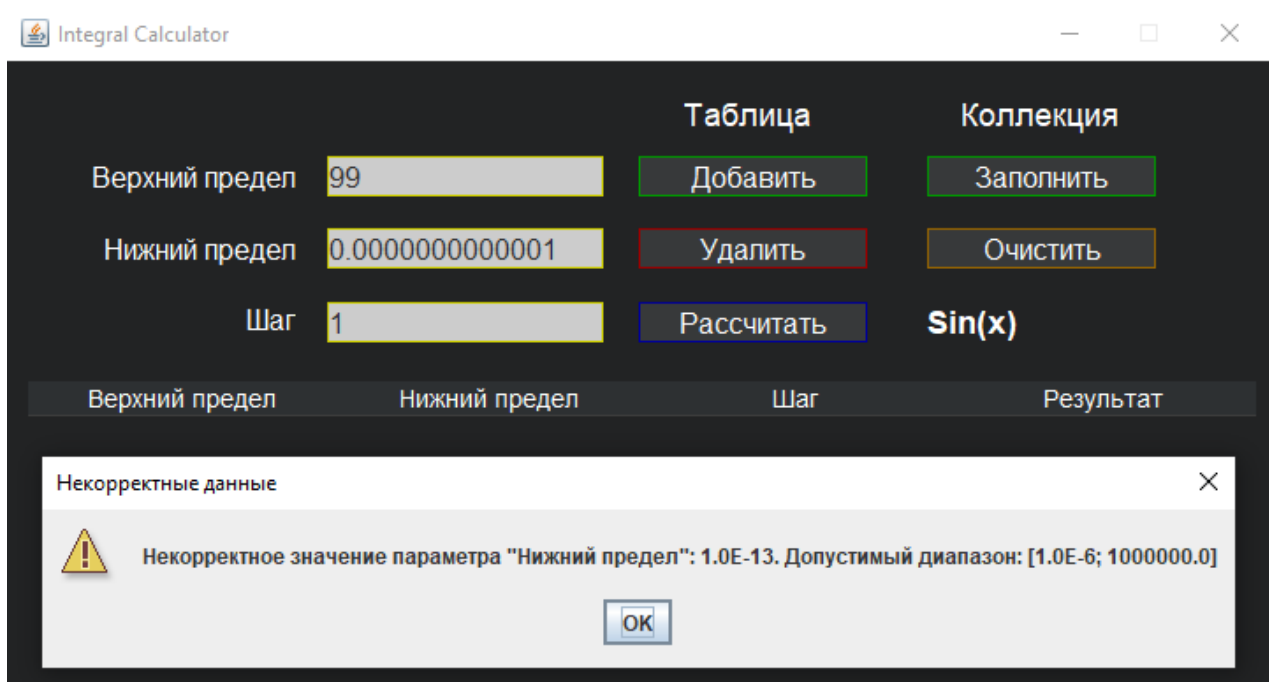
2. Проведём тестирование программы. Попробуем ввести буквы в текстовое поле.



Исключение корректно обработано. Далее попробуем превысить допустимый числовой предел.



Исключение корректно обработано. Далее попробуем ввести число, которое меньше 0,000001.



Все исключения корректно обработаны.

Вывод: в ходе выполнения лабораторной работы были получены навыки работы с обработкой исключительных ситуаций.